

DATA STRUCTURES

Textbook:

*Fundamentals of Data Structure in C++,
second edition, Silicon Press*

Instructor:

吕建华

Email: lujianhua@seu.edu.cn

地 址：计算机楼230

Teaching assistants:

:

@qq.com

Total class hours: 64

Week 1-16: Tuesday : 6-7

Friday : 8-9

Total lab. Hours: 16/16

Week 1-16: Friday : 10

Venue : J3-101

Assignments and Projects:

- Should be handed to
- lujianhua@seu.edu.cn
- **Deadline: In One weeks after assignments**

Assignments and Projects:

•Mail Title

- DS2023_周_日_学号姓名
 - DS2023_1_2_0001张三 (第一周周2)
 - DS2023_1_5_0001张三

•Attached file names

- DS2023_页_题号_学号姓名
 - DS2023_10_2_0001张三 (第10页第2题)

Evaluation:

Course Attendance:	10%,
Exercises and Projects:	30% ,
Discussions:	10% ,
Final Examination (Semi-Open):	50%

References:

- 1 金远平, 数据结构(C++描述), 清华大学出版社, 2005
- 2 T. A. Standish, Data Structures, Algorithms & Software Principles in C, Addison-Wesley Publishing Company, 1994
- 3

Tips

- **Make good use of your time in class**
 - **Listen**
 - **Think**
 - **Take notes**
 - **Practice**
- **Utilize your free time**
 - **Go over**
 - **Programing**
- **Take a pen, a notebook and some loose-leaves**
 - **Notes**
 - **Exercises**

Prerequisites:

Programming Language: C, C++

Pointer in C & C++

Templates

Followup courses:

Advanced Data Structures

高级数据结构 2(2)

**Programming Practice on Data
Structures and Algorithms**

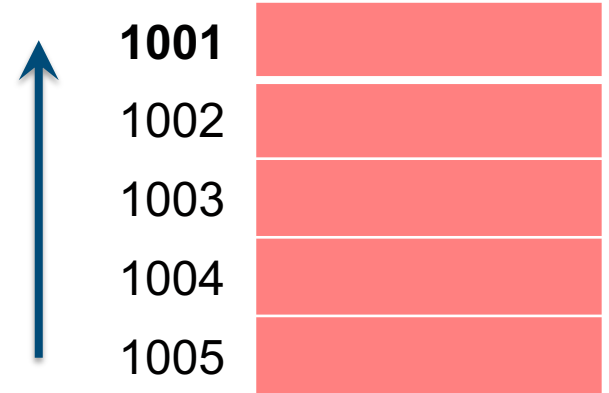
数据结构与算法专题实践 3(1)

Related courses:

So many ...

Q: Memory layout for a 2D array?

Compiler



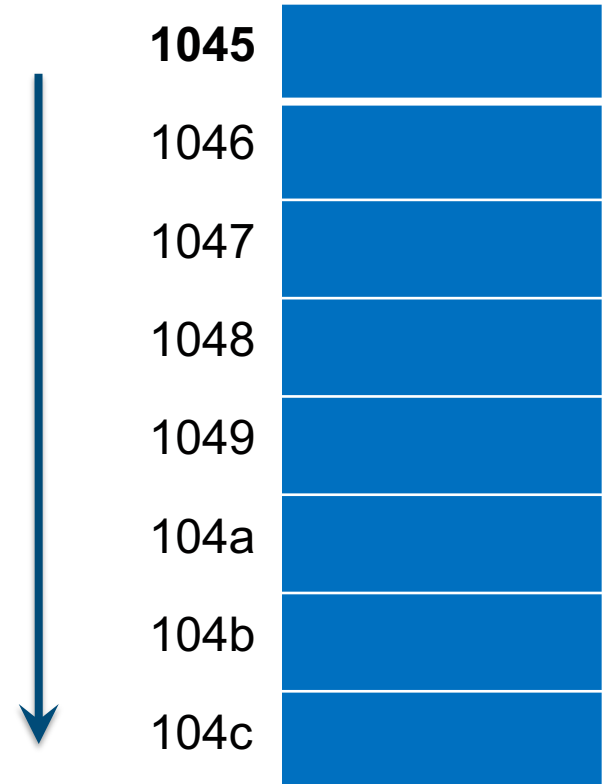
`sizeof(int) :`

?

`sizeof(int*) :`

?

Runtime
New *



```
int a1;  
int * a2 = new int;  
int * a3 = new int();  
int * a4 = new int[4];
```

```
cout << &a1; //&a1:1010  
cout << &a2;  
cout << &a3;  
cout << &a4;
```

Run multiple times...

1001

1002

1003

1004

1005

1045

1046

1047

1048

1049

104a

104b

104c

```
int a1;  
int * a2 = new int;  
int * a3 = new int();  
int * a4 = new int[4];
```

```
cout << a1;  
cout << a2;  
cout << a3;  
cout << a4;
```

Run multiple times...

1001

1002

1003

1004

1005

1045

1046

1047

1048

1049

104a

104b

104c

```
int a1;  
int * a2 = new int;  
int * a3 = new int();  
int * a4 = new int[4];
```

```
cout << a1;  
cout << a2;  
cout << a3;  
cout << a4;  
cout << a4+1; // a4: 1040  
Run multiple times...
```

1001

1002

1003

1004

1005

1045

1046

1047

1048

1049

104a

104b

104c

```
int a1;  
int * a2 = new int;  
int * a3 = new int();  
int * a4 = new int[4];
```

```
cout << *a2;  
cout << *a3;  
cout << *a4;  
cout << *a4+1;  
cout << *(a4+1);
```

1001

1002

1003

1004

1005

1045

1046

1047

1048

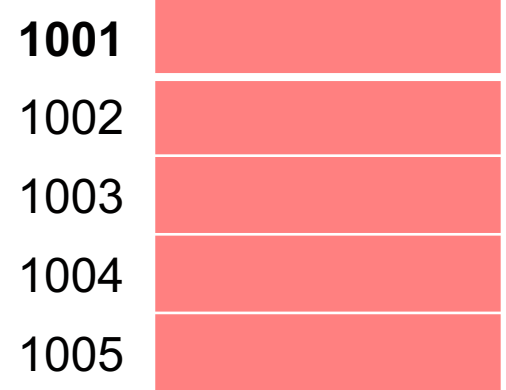
1049

104a

104b

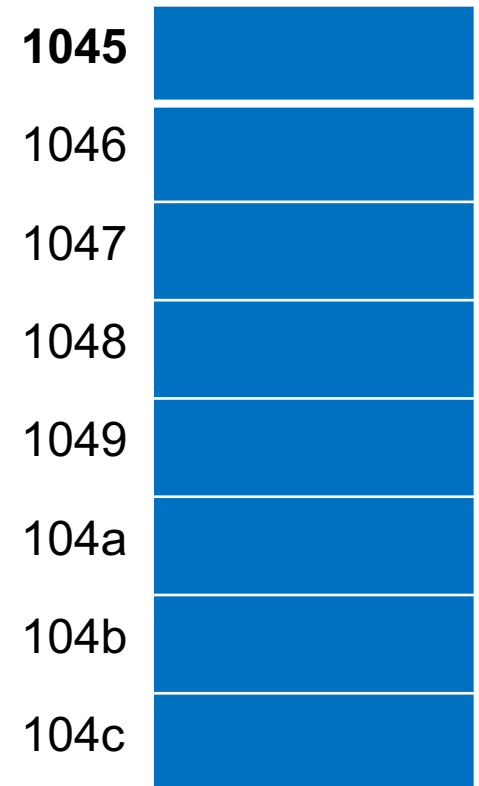
104c


```
Class A {  
public:  
    int * a1;  
    int * a2;  
};
```



```
A ptr_a = new A[4];
```

```
// ptr_a == 1040;  
cout << ptr_a + 1;
```



```
Class A {  
public:  
    int * a1;  
    int * a2;  
};
```

1001

1002

1003

1004

1005

```
A ptr_a = new A[4];
```

```
ptr_a->a1=new int[4];
```

```
ptr_a->a2=new int[6];
```

```
// ptr_a == 1048;
```

```
cout << ptr_a + 1;
```

1045

1046

1047

1048

1049

104a

104b

104c

In Computer science, a **data structure** is a particular way of **storing** and **organizing** data in a computer so that it can be used **efficiently**.

物有本末，事有终始。
知所先后，则近道矣。

Basic Concepts

Purpose:

Provide the tools and techniques necessary to design and implement **large-scale software systems**, including:

- Data abstraction and encapsulation
- Algorithm specification and design
- Performance analysis and measurement
- Recursive programming

Overview: System Life Cycle

(1) Requirements

specifications of purpose

input

output

(2) Analysis

break the problem into manageable pieces

bottom-up

top-down

Overview: System Life Cycle

(3) Design

a SYSTEM? (from the designer's angle)

data objects

operations on them

TO DO

abstract data type

algorithm specification and design

Example: scheduling system of university

??

??

(4) Refinement and coding

**representations for data object
algorithms for operations
components reuse**

(5) Verification and maintenance

**correctness proofs
testing
error removal
update**

Data Abstraction and Encapsulation

Data Abstraction is the separation between the *specification* of a data object and its *implementation*.

Data Encapsulation or Information Hiding is the *concealing* of the implementation details of a data object from the outside world.

Text Book: **DVD example.**

Idol Industry
S. Korea Magic

A Data Type is a collection of *objects* and a set of *operations* that act on those objects.

predefined and user-defined:

char, int, arrays, structs, classes.

An Abstract Data Type (ADT) is a data type with the **specification** of the objects and the **specification** of the operations **on the objects** being separated from the **representation** of the objects and the **implementation** of the operations.

Benefits of data abstraction and data encapsulation:

(1) Simplification of software development

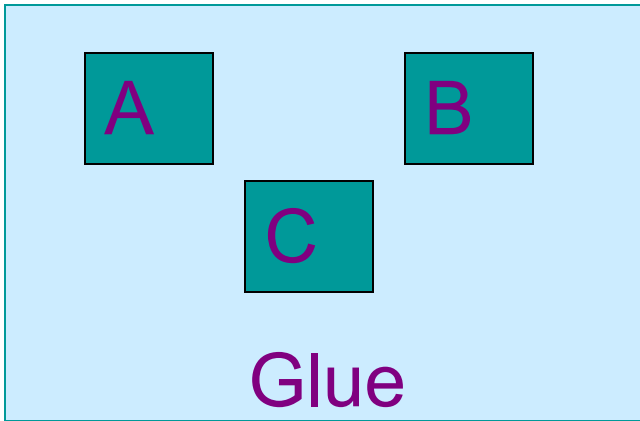
Applicaton : data types **A, B, C** & Code **Glue**

(a) a team of 4 programmers

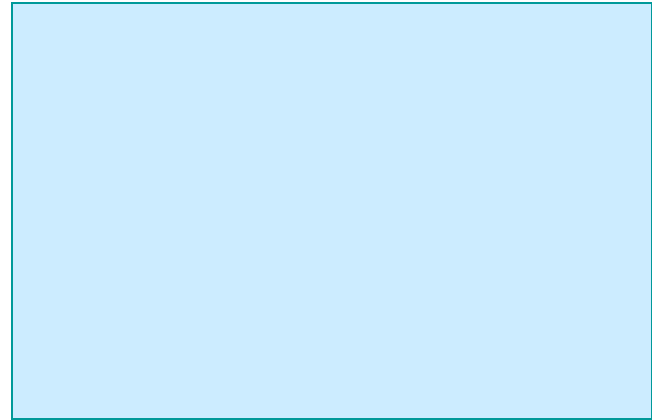
(b) a single programmer

(2) Testing and debugging

Code with data abstraction



Code without data abstraction



Unshaded areas represent code to be searched for bugs.

(3) Reusability

data structures implemented as distinct entities of a software system

(4) Modifications to the representation of a data type

a change in the internal implementation of a data type will not affect the rest of the program as long as its interface does not change.

Algorithm Specification

An **algorithm** is a finite set of instructions that, if followed, accomplishes a particular task.

Must satisfy the following criteria:

- (1) Input** Zero or more quantities externally supplied.
- (2) Output** At least one quantity is produced.
- (3) Definiteness** Clear and unambiguous.
- (4) Finiteness** Terminates after a finite number of steps.
- (5) Effectiveness** Basic enough, feasible

Compare: algorithms and programs

Finiteness

Recursion

Exercises: P32-2 , P33-14

Performance Analysis and Measurement

Definition:

The **Space complexity** of a program is the amount of memory it needs to run to completion.

The **Time complexity** of a program is the amount of computer time it needs to run to completion.

(1) Priori estimates --- **Performance analysis**

(2) Posteriori testing--- **Performance measurement**

Performance Analysis

Space complexity

The space requirement of program P:

$$S(P) = c + S_P(\text{instance characteristics})$$

We concentrate solely on S_P .

Performance Analysis

Example 1.10

```
float Rsum (float *a, const int n) //compute  $\sum_{i=0}^{n-1} a[i]$   
recursively  
{  
    if (n <=0) return 0;  
    else return (Rsum(a,n-1)+a[n-1]);  
}
```

The instances are characterized by

n

each call requires 4 words (n, a, return value, return address)

the depth of recursion is

n+1

$S_{\text{rsum}}(n) =$

$4(n+1)$

Time complexity

Run time of a program P:

$$T(P) = c + t_p(\text{instance characteristics})$$

A **program step** is loosely defined as a syntactically or semantically meaningful segment of a program that has an execution time that is **independent** of instance characteristics.

In P41-43 of the textbook, there is an detailed assignment of step counts to statements in C++.

Step Count

A step is an amount of computing that does not depend on the instance characteristic n

10 adds, 100 subtracts, 1000 multiplies
can all be counted as a single step

$n/10000$ adds cannot be counted as 1 step

Our main concern:

how many steps are needed by a program to solve a particular problem instance?

2 ways:

(1) count

(2) table

Example 1.12

```
count=0;
float Rsum (float *a, const int n)
{
    count++; // for if
    if (n <=0) {
        count++; // for return
        return 0;
    }
    else {
        count++; // for return
        return (Rsum(a,n-1)+a[n-1]);
    }
}
```

$$t_{\text{Rsum}}(0) = 2,$$

$$t_{\text{Rsum}}(n) = 2 + t_{\text{Rsum}}(n-1) \\ = 2 + 2 + t_{\text{Rsum}}(n-2)$$

·

·

·

$$= 2n + t_{\text{Rsum}}(0)$$

$$= 2n + 2$$

Examp Let us use a table to count its total steps.

1 void F

2 { // cc

3 if (n

4 else

5 in

6 fo

7 {

8

9

10

11 } /

12 c

13 } //

14 }

Line	s/e	frequency	total steps
1	0	1	0
2	0	1	0
3	1 ($n > 1$)	1	1
4	0	1	0
5	2	1	2
6	1	n	n
7	0	n-1	0
8	1	n-1	n-1
9	1	n-1	n-1

10	1	$n-1$	$n-1$
11	0	$n-1$	0
12	1	1	1
13	0	1	0
14	0	1	0

So

for $n > 1$, $t_{\text{Fibonacci}}(n) = 4n + 1$,

for $n = 0$ or 1 , $t_{\text{fibonacci}}(n) = 2$

Sometime, the instance characteristics is related with the content of the input data set.

e.g., BinarySearch.

Hence:

- **best-case**
- **worst-case,**
- **average-case.**

Asymptotic Notation

Because of the inexactness of what a step stands for, we are mainly concerned with the magnitude of the number of steps.

Definition [O]: $f(n)=O(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \leq c g(n)$ for all $n, n>n_0$.

Example 1.13: $3n+2=O(n)$, $6*2^n+n^2=O(2^n), \dots$

Note $g(n)$ is an **upper bound**.

$n = O(n^2)$, $n = O(2^n)$, ...,

for $f(n) = O(g(n))$ to be **informative**, $g(n)$ should be
as small as possible.

In practice, the coefficient of $g(n)$ should be 1. We never say $O(3n)$.

Theory 1.2: if $f(n)=a_m n^m+\dots+a_1 n+a_0$, then $f(n)=O(n^m)$.

When the complexity of an algorithm is actually, say, $O(\log n)$, but we can only show that it is $O(n)$ due to the limitation of our knowledge, it is OK to say so. This is one benefit of O notation as upper bound.

Self-study:

Ω --- low bound

Θ --- equal bound

A Few Comparisons

Function #1

Function #2

$$n^3 + 2n^2$$



$$100n^2 + 1000$$

$$n^{0.1}$$



$$\log n$$

$$n + 100n^{0.1}$$



$$2n + 10 \log n$$

$$5n^5$$



$$n!$$

$$n^{-15}2^n/100$$



$$1000n^{15}$$

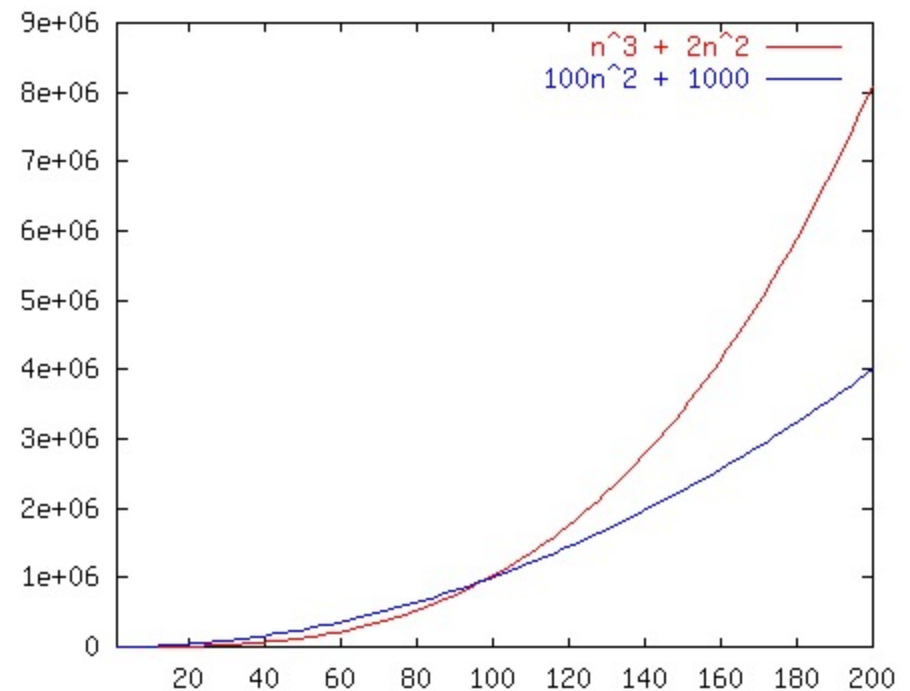
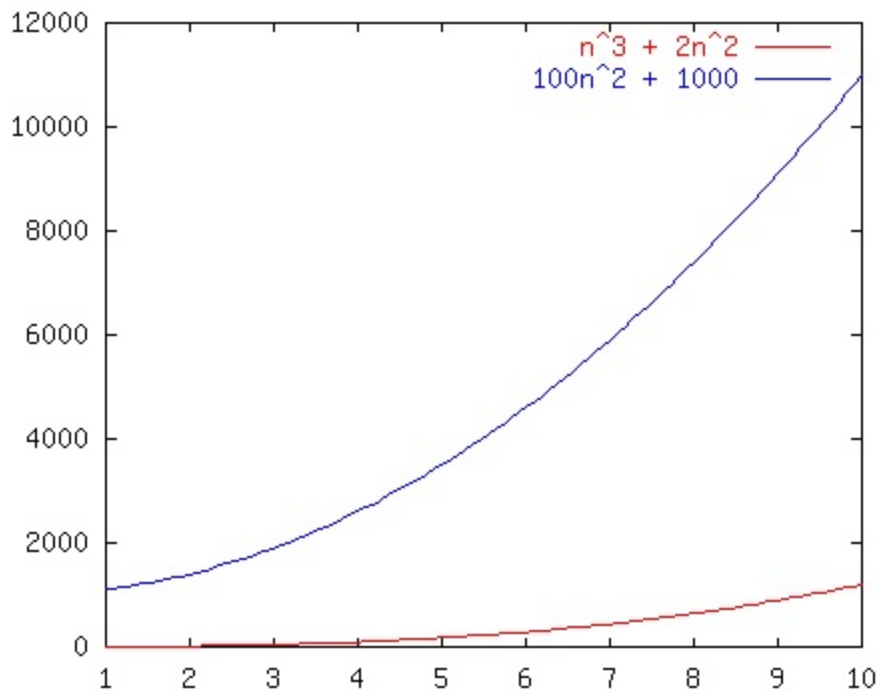
$$8^{2\log n}$$



$$3n^7 + 7n$$

Race I

$n^3 + 2n^2$ vs. $100n^2 + 1000$

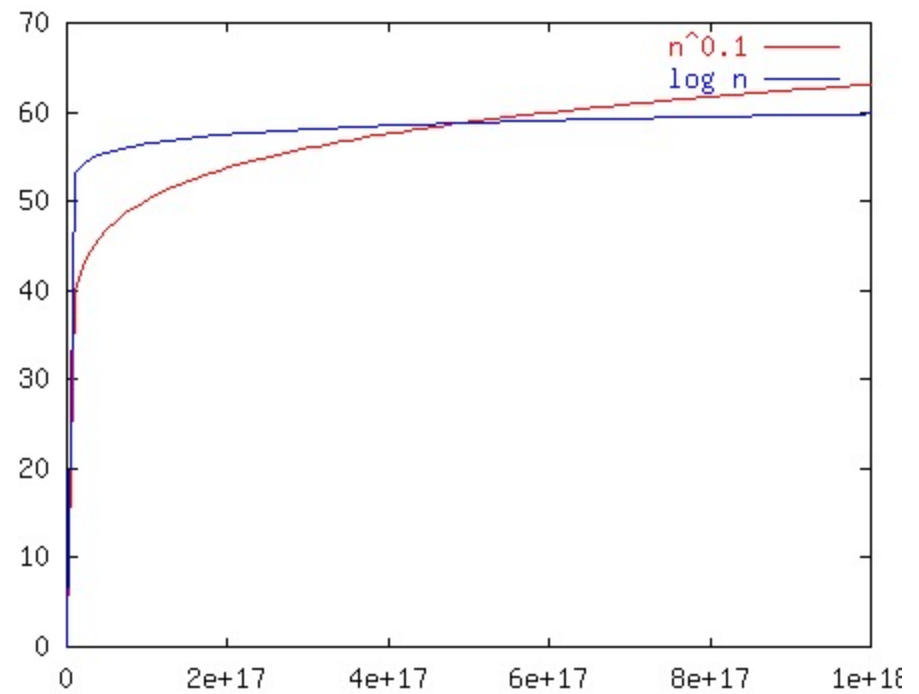
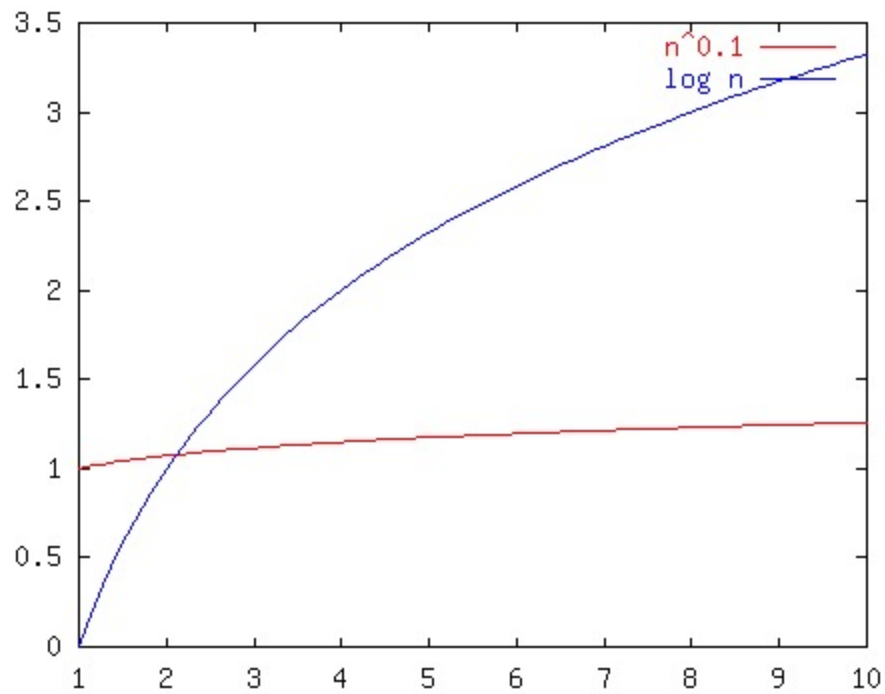


Race II

$n^{0.1}$

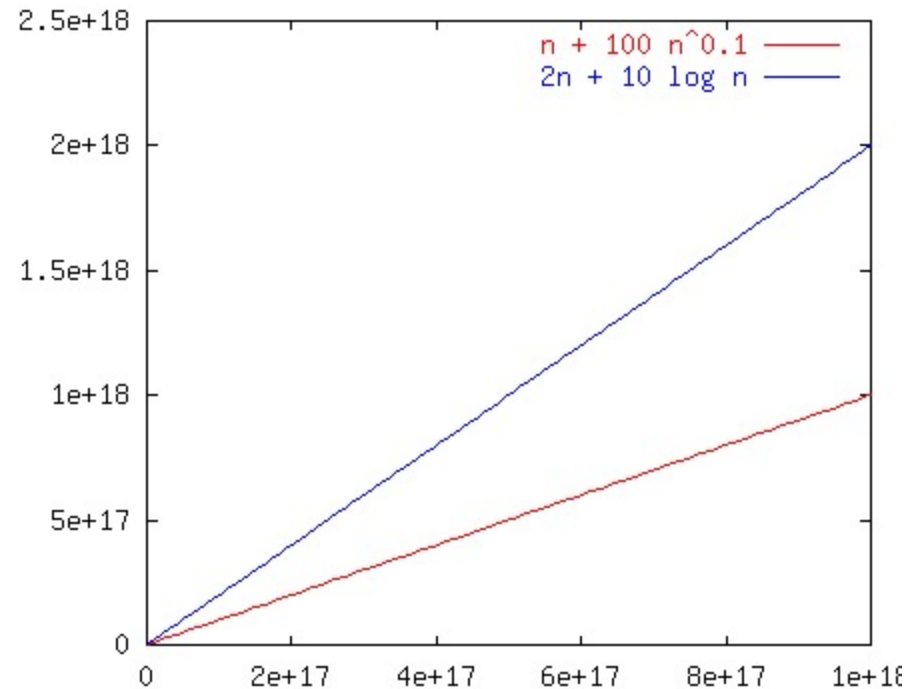
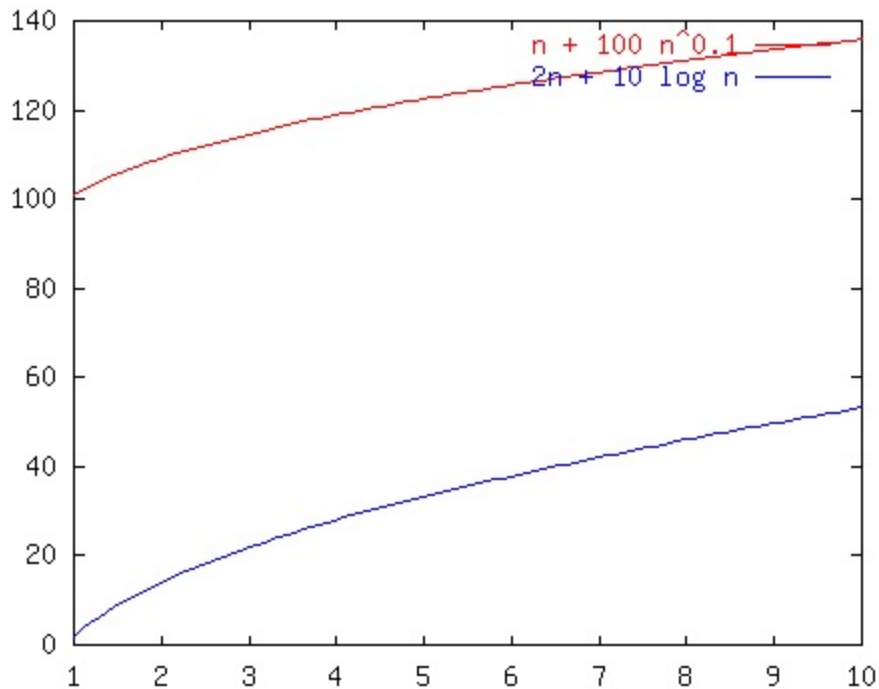
vs.

$\log n$



Race III

$n + 100n^{0.1}$ vs. $2n + 10 \log n$

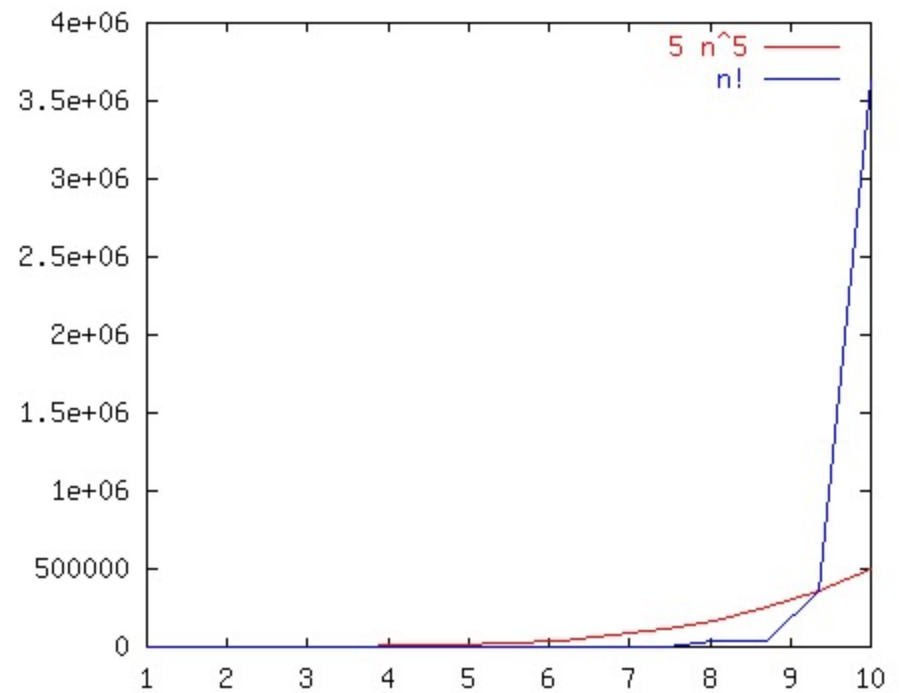
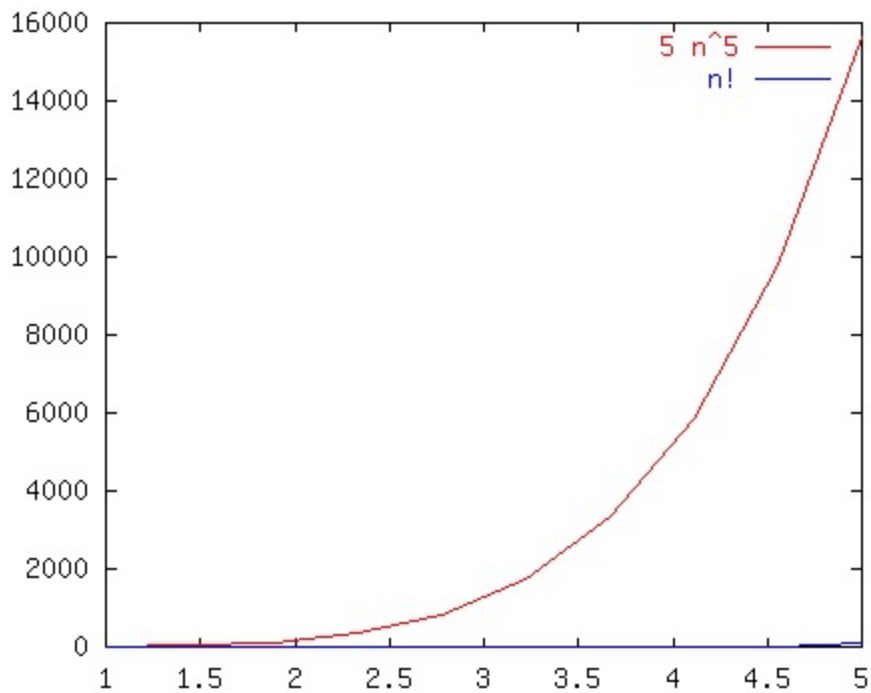


Race IV

$5n^5$

vs.

$n!$

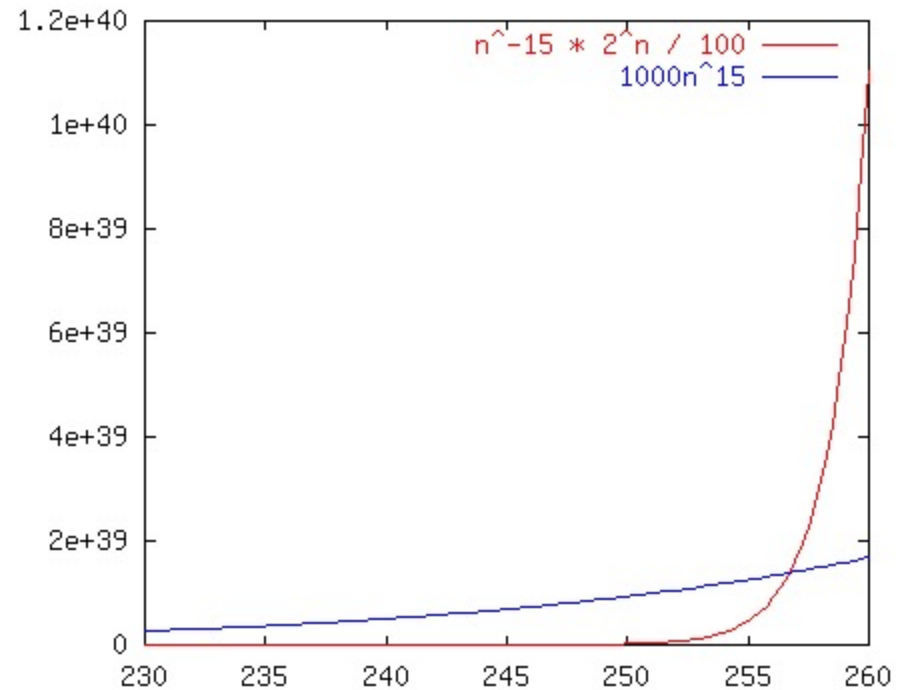
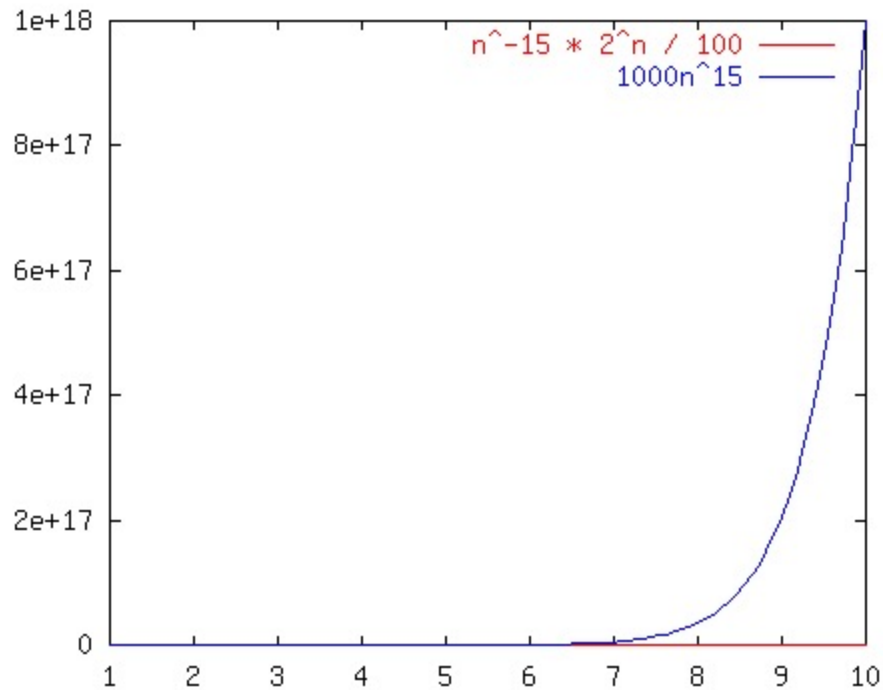


Race V

$$n^{-15} 2^n / 100$$

vs.

$$1000n^{15}$$

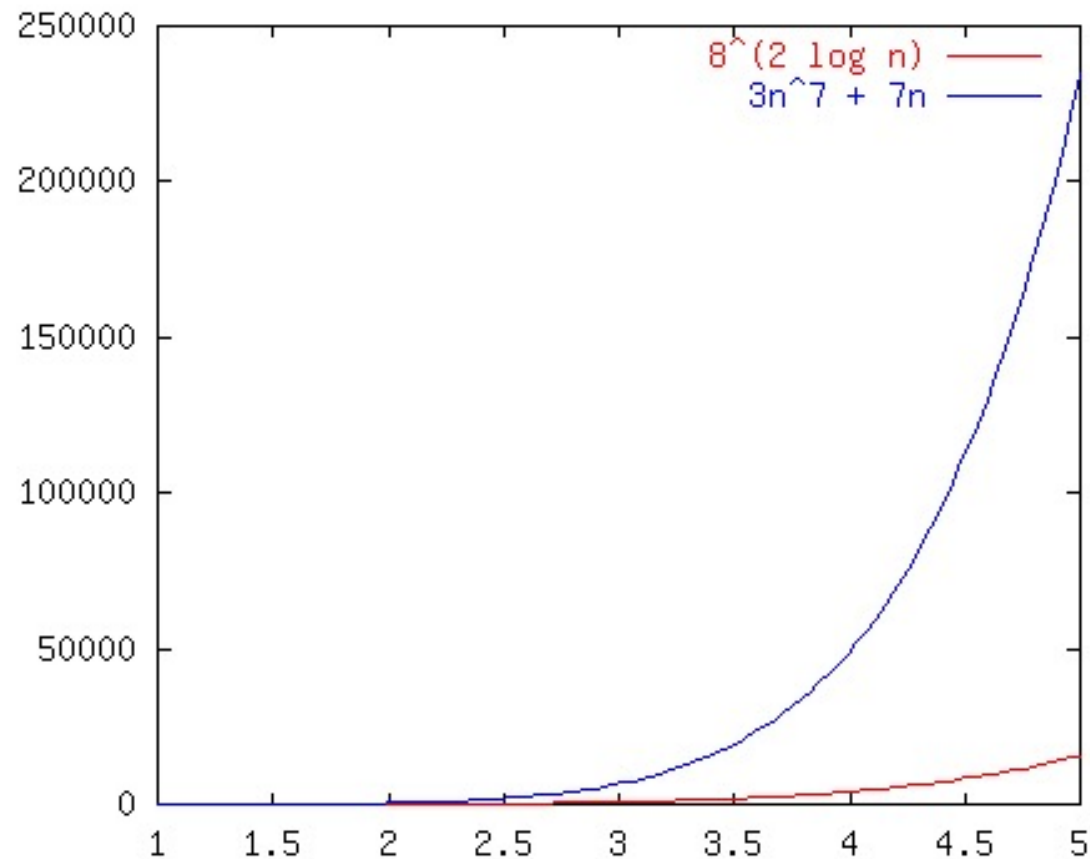


Race VI

$$8^{2\log(n)}$$

vs.

$$3n^7 + 7n$$



The Losers Win

Function #1

Function #2

Better algorithm!

$$n^3 + 2n^2$$

$$100n^2 + 1000$$

$$O(n^2)$$

$$n^{0.1}$$

$$\log n$$

$$O(\log n)$$

$$n + 100n^{0.1}$$

$$2n + 10 \log n$$

$$\mathbf{TIE} \ O(n)$$

$$5n^5$$

$$n!$$

$$O(n^5)$$

$$n^{-15} 2^n / 100$$

$$1000n^{15}$$

$$O(n^{15})$$

$$8^{2 \log n}$$

$$3n^7 + 7n$$

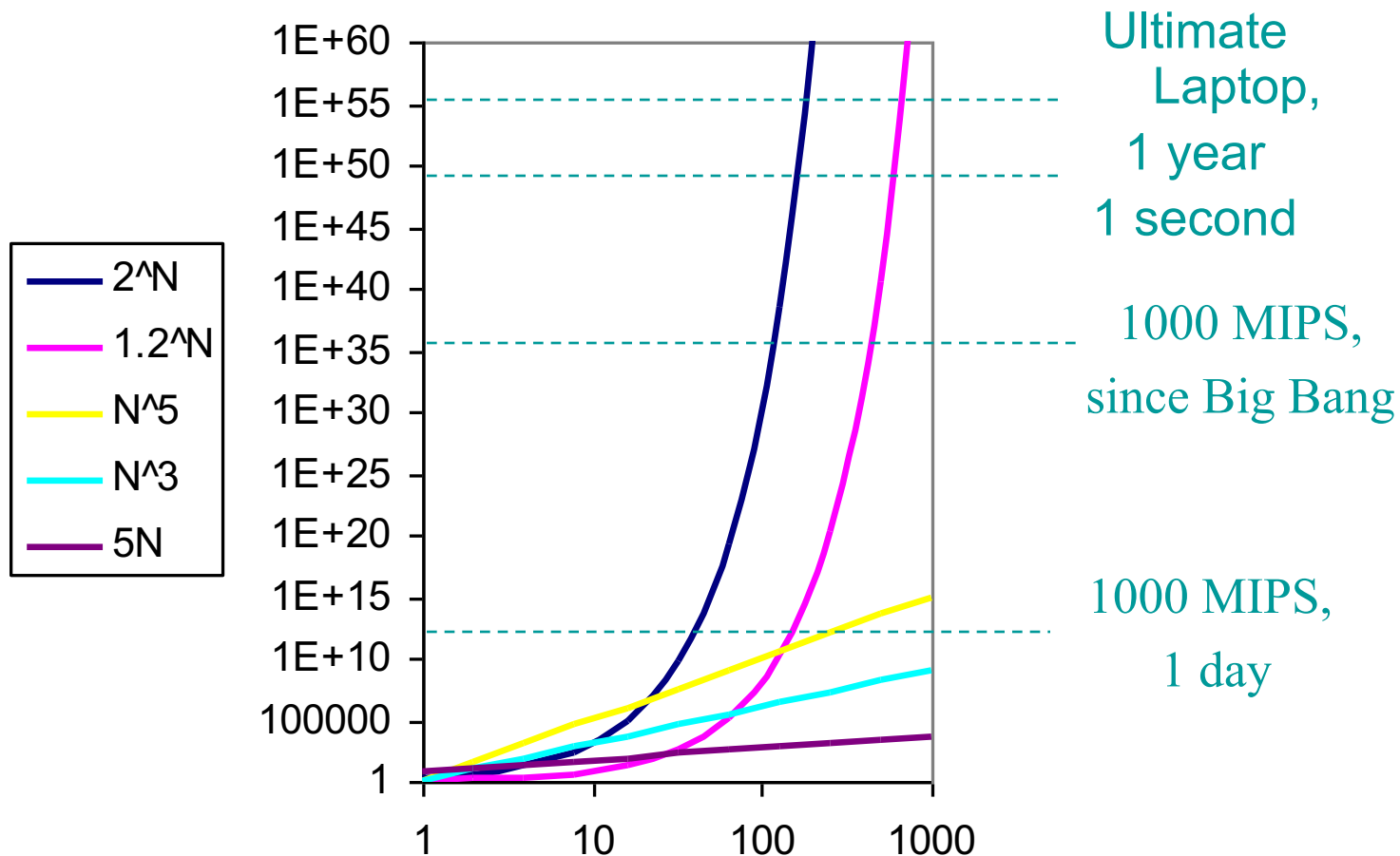
$$O(n^6)$$

Common Names

constant:	$O(1)$	
logarithmic:	$O(\log n)$	
linear:	$O(n)$	
log-linear:	$O(n \log n)$	
quadratic:	$O(n^2)$	
polynomial:	$O(n^k)$	(k is a constant)
exponential:	$O(c^n)$	(c is a constant > 1)

Practical Complexity

How the various functions grow with n?



n	$f(n)=n$	$f(n)=n\log_2 n$	$f(n)=n^2$	$f(n)=n^4$	$f(n)=n^{10}$	$f(n)=2^n$
10	.01 μ s	.03 μ s	.1 μ s	10 μ s	10s	1 μ s
20	.02 μ s	.09 μ s	.4 μ s	160 μ s	2.84h	1 ms
30	.03 μ s	.15 μ s	.9 μ s	810 μ s	6.83d	1 s
40	.04 μ s	.21 μ s	1.6 μ s	2.56ms	121d	18m
50	.05 μ s	.28 μ s	2.5 μ s	6.25ms	3.1y	13 d
100	.1 μ s	.66 μ s	10 μ s	100 ms	3171y	4*10 ¹³ y
10 ³	1 μ s	9.66 μ s	1ms	16.67m		
10 ⁴	10 μ s	130 μ s	100ms	115.7d		
10 ⁵	100 μ s	1.66ms	10s	3171y		

Table 1.8: Times on a 1-billion-steps-per-second computer

Performance Measurement

Performance measurement is concerned with obtaining the **actual** space and time requirements of a program.

To time a short event it is necessary to **repeat** it several times and divide the total time for the event by the number of repetitions.

Let us look at the following program:

```
int SequentialSearch (int *a, const int n, const int x )  
{ // Search a[0:n-1].  
    int i;  
    for (i=0; i < n && a[i] != x; i++;)  
        if (i == n) return -1;  
    else return i;  
}
```

```

void TimeSearch ( )
{
    int a[1000], n[20];
    const long r[20] = {300000, 300000, 200000, 200000,
        100000, 100000, 100000, 80000, 80000, 50000, 50000,
        25000, 15000, 15000, 10000, 7500, 7000, 6000, 5000,
        5000 };

    for ( int j=0; j<1000; j++ ) a[j] = j+1; //initialize a
    for ( j=0; j<10; j++ ) { //values of n
        n[j] = 10*j; n[j+10] = 100*(j+1 );
    }

    cout << “ n    total    runTime” << endl;

```

```

for ( j=0; j<20; j++ ) {
    long start, stop;
    time (&start);                                // start timer
    for ( long b=1; b<=r[j]; b++ )
        int k = seqsearch(a, n[j], 0 ); //unsuccessful search
    time (&stop);                                // stop timer
    long totalTime = stop - start;
    float runTime = (float) (totalTime) / (float)(r[j]);
    cout << " " << n[j] << " " << totalTime << " " << runTime
        << endl;
    }
}

```

The results of running *TimeSearch* are as in the next slide.

n	total	runTime	n	total	runTime
0	241	0.0008	100	527	0.0105
10	533	0.0018	200	505	0.0202
20	582	0.0029	300	451	0.0301
30	736	0.0037	400	593	0.0395
40	467	0.0047	500	494	0.0494
50	565	0.0056	600	439	0.0585
60	659	0.0066	700	484	0.0691
70	604	0.0075	800	467	0.0778
80	681	0.0085	900	434	0.0868
90	472	0.0094	1000	484	0.0968

Times in hundredths of a second, the plot of the data can be found in Fig. 1.7.

Issues to be addressed:

- (1) Accuracy of the clock**
- (2) Repetition factor**
- (3) Suitable test data for worst-case or average performance**
- (4) Purpose: comparing or predicting?**
- (5) Fit a curve through points**

Exercises:

P69-3,4

Experiment:

P69-10


```
x=2;  
while (x<n/2)  
    x=2*x;
```

```
d=1; x = 0;  
while (x<n) {  
    d ++;  
    x = x + d;  
}
```

```
int f(int n)
{
    if (n<=1) return n;
    return n+f(n-1);
}
```

```
long F(int N)
{
    return N < 2 ?
        N : F(N-1)+F(N-2);
}
```